

Git Installation and Setup Guide

Release 1.0
October, 2023



A SAMPLE INSTALLATION GUIDE
AUTHORED BY **AISHWARYA M**

Contents

List of Tables.....	iii
Abstract.....	4
Notice.....	5
Copyright License.....	6
Trademarks.....	6
Preface.....	7
1. Introduction.....	8
1.1. Git vs. GitHub™.....	8
1.2. Prerequisites.....	9
1.2.1. Supported Operating System.....	9
1.2.2. Internet Connection.....	10
1.2.3. Administrative Privileges.....	10
2. Installing Git on Various OS.....	11
2.1. Windows.....	11
2.2. Ubuntu.....	12
2.3. macOS.....	13
2.4. Linux.....	14
3. Creating a GitHub Account.....	15
4. Configuring Git.....	16
5. SSH Key for GitHub.....	17
6. Sample Git Commands.....	19
Index.....	a

List of Tables

Table 1: The difference between Git and GitHub.....	8
Table 2: Supported OS.....	9
Table 3: Sample Git commands.....	19

Abstract

A brief summary of installing and setting up Git and GitHub.

This Git, GitHub installation and setup guide provides detailed, step-by-step instructions for installing Git and configuring it on various operating systems, including Windows 7 to Windows 11, Ubuntu, Linux and macOS. Starting with the installation process, the guide addresses specific components and configurations for each platform, ensuring a smooth and tailored experience for users. Additionally, the guide covers the essential steps of creating a GitHub account and establishing a secure SSH key for interaction with the GitHub platform.

Notice

A sample installation guide offering detailed instructions for Git and GitHub™ setup, prepared by Aishwarya M for portfolio purposes.

This installation and setup guide is intended to provide detailed instructions for installing Git and configuring it on various OS. The guide aims to assist users in the installation and setup process for Git and GitHub™. It is important to note that software versions, installation methods, and user interfaces may evolve over time, and the instructions provided are based on the state of Git and GitHub™ as of August 26, 2023. Users are encouraged to refer to the official documentation and resources for the latest updates and additional information.

Copyright License

Any unauthorized reproduction or distribution of this user guide may result in copyright infringement and is strictly prohibited.

(c) Copyright Git, GitHub™ 2023 All Rights Reserved.

This user guide, including all its contents and any associated material, is protected by copyright law and is owned by Git and GitHub™. All rights are reserved. No part of this user guide may be reproduced, distributed, or transmitted in any form or by any means, including printing, photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the company. The information contained in this user guide is for informational purposes only and is subject to change without prior notice. Git, GitHub™ and its affiliates assume no responsibility for any errors or omissions in the information contained in this user guide.

Trademarks

GitHub™ is a registered trademark of GitHub, Inc.

Git is an open-source distributed version control system developed by Linus Torvalds.

Preface

Welcome to Git and GitHub™, where collaboration and version control are streamlined for your development projects. With Git and GitHub™, you can efficiently manage your codebase, collaborate with ease, and track changes effortlessly. Whether you're a developer looking to streamline your workflow or a team working on a project, Git and GitHub™ have the tools you need. This guide is crafted to empower you with the knowledge and skills.

1. Introduction

Git is a distributed version control system used for tracking changes in source code. This guide provides detailed, step-by-step instructions for installing Git on different operating systems, configuring it, creating a GitHub account, and setting up an SSH key for secure access to GitHub.

1.1. Git vs. GitHub™

Table 1. The difference between Git and GitHub

Attribute	Git	GitHub
Definition	A distributed version control system for tracking changes in source code.	A web-based platform that provides hosting for Git repositories and collaboration tools.
Core Purpose	To manage and track changes in code at the local level.	To facilitate collaboration, remote hosting, and project management using Git repositories.
Location	Installed and operates on your local computer.	Hosted on the internet as a cloud-based service.
Usage	Developers use Git for version control within their own projects and collaborate locally.	Developers and teams use GitHub for remote collaboration, sharing, and backup of Git repositories.
Internet Access	Not dependent on internet access; operates locally.	Requires an internet connection for remote repository access and collaboration.
Repositories	Git repositories are local (on your computer) and can be pushed to remote locations.	Hosts remote Git repositories that can be accessed by multiple users from various locations.

Table 1. The difference between Git and GitHub (continued)

Attribute	Git	GitHub
Ownership	Developers own and maintain their local Git repositories.	Allows users to create and manage remote Git repositories, which can be owned by individuals or organizations.
Collaboration	Collaboration in Git is primarily local or within a network.	Facilitates collaboration among distributed teams by providing tools for code review, issue tracking, and project management.
Examples	Git is used via the command line. Some examples are, <pre>\$ git init</pre> <pre>\$ git config --global user.name "Your Name"</pre> <pre>\$ git clone <repository_url></pre>	GitHub provides a web interface for managing repositories, issues, and pull requests, in addition to Git commands.

1.2. Prerequisites

It is essential to ensure that the below prerequisites are met for successful installation and set up.

1.2.1. Supported Operating System

Table 2. Supported OS

S. No.	Supported OS	Supported Versions
1.	Windows	Windows 7 and above

Table 2. Supported OS (continued)

S. No.	Support- ed OS	Supported Versions
2.	Ubuntu	Git is compatible with various Ubuntu versions, including Ubuntu 16.04 LTS, Ubuntu 18.04 LTS, Ubuntu 20.04 LTS.
3.	macOS	Git is supported by macOS High Sierra (10.13) and later, which also includes macOS Mojave, macOS Catalina, macOS Big Sur.

1.2.2. Internet Connection

A reliable internet connection is crucial for downloading Git and connecting to GitHub™. You'll need internet access to download Git's installation package and create a GitHub account, which is a pivotal step in the process. A stable connection ensures that you can access the latest updates and collaborate with other developers via GitHub.

1.2.3. Administrative Privileges

Depending on your operating system and specific setup, you may need administrative privileges to install Git. Administrative privileges grant you the necessary permissions to make system-level changes during installation. On Windows, for example, administrative privileges are typically required for system-wide installations. It's important to have these privileges or consult your system administrator to facilitate the installation process smoothly.

2. Installing Git on Various OS

Now, let's delve into the detailed installation process for Git on various operating systems.

2.1. Windows

To install Git on your local system for Windows 7 and above, follow the steps provided below,

1. Visit the official Git website - [Git for Windows](#)
2. Click the "Download" button to initiate the .exe file download process
3. Locate the downloaded executable application file on your system
4. Run the installer and Configure Components.
 - When you run the Git installer, you will encounter a step that allows you to customize the components you want to install. This step is crucial for tailoring your Git installation to your specific needs. You might notice an option called "**Git LFS**," which stands for Large File Storage, checked.



Attention: Git LFS is an extension to Git that enables it to handle large files more efficiently. When you work with binary files like images, videos, or other non-textual data, Git LFS helps manage them by storing them outside the regular Git repository, reducing the repository's size and improving performance.



Note: In most cases, the Git LFS option is selected by default in the installer. Unless you have specific requirements and are certain you won't need Git LFS for your projects, it's generally advisable to leave this option selected during installation. Even if you don't anticipate immediately using large files in your repositories, having Git LFS installed won't impact your regular Git usage.

5. Select your default text editor. Git uses a text editor for messages and commit descriptions. You can select your preferred text editor. Some common choices include **Visual Studio Code**, **Vim**, **Emacs** and **Notepad++**, **Nano**.
6. Choose your default branch name. While installing Git on Windows, you can specify your preferred default branch name. By default, Git has historically used "**master**" as the main branch name.
7. Select the Terminal Emulator. The installer offers various options for terminal emulators. To ensure a consistent and Git-friendly experience, it's recommended to select "**Use Git Bash only**." This choice aligns with best practices for Git usage on Windows.
8. Configure Line Ending Conversions. This helps in managing line endings in the Git repository. For optimal cross-platform compatibility, it's advised to stick with the default setting, which is "**Checkout Windows-style, commit Unix-style line endings**."
9. After configuring the components and settings according to your preferences, click the "**Install**" button to commence the installation process.

2.2. Ubuntu

To begin the installation process, open a terminal on your Ubuntu system by searching for "**Terminal**" in the applications menu. You can also use the keyboard shortcut `Ctrl + Alt + T` to open a terminal.

1. Before installing any software, it is a good practice to update the package list to ensure you have the latest information about available packages. Execute the following command to check if you have the recent version of Git:

```
$ sudo apt update
```



Note: *apt update*: This command will connect to Ubuntu's software repositories and fetch the latest information about available packages, ensuring your package manager knows about the most recent versions of Git and its dependencies.

2. You will be prompted to type in your sudo password. Once the password is entered, press "Enter" on the keyboard.
3. Once the package list is updated, you can proceed to install Git using the package manager. Execute the following command:

```
$ sudo apt install git
```

4. The package manager will calculate the necessary dependencies and confirm that you want to install Git. Type "Y" for Yes and press Enter to confirm. As soon as the installation process begins, you will see a progress indicator showing the download and installation of Git and its associated packages.
5. To ensure that Git has been successfully installed, you can check the Git version by running the following command:

```
$ git --version
```

This command will display the installed Git version, confirming that Git is now available and operational on your Ubuntu system.

2.3. macOS

Git is often pre-installed on macOS, if not go to <https://git-scm.com/download/mac>. To verify and install if needed, follow these steps:

1. Open the Terminal application.
2. Type the following command and press Enter:

```
$ git --version
```

If Git is installed, you'll see the version information. If not, the Terminal will prompt you to install the Xcode Command Line Tools, which include Git.

3. The dialog will state that the 'git' command requires the 'Xcode Command Line Tools' and present two options: "Install" or "Cancel." If prompted, choose to install the Xcode Command Line Tools.
4. You'll be asked to enter your macOS user password to grant administrative access for the installation. Provide your password and press Enter.
5. macOS will start downloading and installing the Xcode Command Line Tools package. This process may take a few minutes.

6. After successful installation, you'll receive a confirmation message in the Terminal, indicating that the installation is complete.
7. To confirm that Git is now available, execute the following command in the Terminal. If Git is installed correctly, it will display the installed Git version.

```
$ git --version
```

Git is now installed on your macOS system. You can start using Git for version control and other development tasks.

2.4. Linux

To install Git for Linux, go to <https://git-scm.com/download/linux>.

If you want to install the basic Git tools on Linux via a binary installer, you can generally do so through the package management tool that comes with your distribution. If you're on Fedora (or any closely-related RPM-based distribution, such as RHEL or CentOS), you can use `dnf`:

```
$ sudo dnf install git-all
```

If you're on a Debian-based distribution, such as Ubuntu, try `apt`:

```
$ sudo apt install git-all
```

3. Creating a GitHub Account

To create a GitHub account, follow the steps given below:



Note: If you already have an existing GitHub account, please skip to the next chapter Configuring Git

1. Visit the GitHub website at <https://github.com>
2. Click the "**Sign Up**" button on the GitHub homepage.
3. Enter your desired username in the "**Username**" field. If the entered user name is not available, GitHub will provide you with several available ones for you to choose from.
4. Provide a valid email address in the "**Email address**" field.
5. Create a strong, unique password for your GitHub account.

Passwords should follow the below criteria to be secured,

- At least 12 characters long but 14 or more is better.
 - A combination of uppercase letters, lowercase letters, numbers, and symbols.
 - Not a word that can be found in a dictionary or the name of a person, a part of the registered e-mail ID, name of a character, product, or organization.
 - Example: JpW&LCyl{5SO
6. Complete the CAPTCHA to confirm you're not a robot.
 7. Choose the "**Free**" plan unless you need the features of a paid plan
 8. Verify your email address by clicking the link sent to your inbox.



Optional: Set up your profile by adding a profile picture and basic information.

9. Indicate your experience level with Git and GitHub. Your GitHub account is created.



Optional: Download and install GitHub Desktop for a graphical interface.

4. Configuring Git

After installing Git, it's important to configure it with your name and email. This configuration helps Git associate your identity with the changes you make in a Git repository, which is important for tracking contributions and collaboration. Please follow the below steps to successfully configure Git,

1. Launch a terminal on your operating system (e.g., Command Prompt on Windows, Terminal on macOS/Linux).
2. Use the following command to set your name, replacing "Your Name" with your actual name:

```
$ git config --global user.name "Your Name"
```

3. Use the following command to set your email, replacing "your.email@example.com" with your actual email address:

```
$ git config --global user.email "your.email@example.com"
```

4. To verify your configuration, enter:

```
$ git config --list
```

This will display your Git configuration details, including your name and email.

5. SSH Key for GitHub

To set up an SSH Key (Secure Shell) for GitHub, follow the below steps:

1. Open a Terminal (Linux/macOS) or Git Bash (Windows).
2. Execute the following command to check for existing SSH Keys:

```
$ ls -al ~/.ssh
```

3. Generate a new SSH key:

```
ssh-keygen -t rsa -b 4096 -C "your_email@example.com"
```

Replace "your_email@example.com" with your GitHub-associated email address.

4. Start the SSH agent:

```
eval "$(ssh-agent -s)"
```

5. Add your SSH key to the SSH agent:

```
ssh-add ~/.ssh/id_rsa
```

6. Copy your SSH public key to the clipboard:

- a. On Linux (using xclip):

```
sudo apt-get install xclip  
xclip -sel clip < ~/.ssh/id_rsa.pub
```

- b. On Windows Subsystem for Linux (WSL):

```
cat ~/.ssh/id_rsa.pub | clip
```

7. Add SSH key to your GitHub account:

- a. Go to **GitHub** and log in.
- b. In the top right corner, click your profile picture and select "**Settings.**"
- c. In the left sidebar, click "**SSH and GPG keys.**"
- d. Click the "**New SSH key**" button.
- e. Provide a title for your SSH key and paste the key in the "**Key**" field.
- f. Click the "**Add SSH key**" button and confirm your GitHub password when prompted.

8. Test your SSH connection to GitHub:

```
ssh -T git@github.com
```

9. You're done. Your SSH key is now set up for GitHub authentication, allowing you to interact with GitHub repositories securely.

6. Sample Git Commands

Table 3. Sample Git commands

Command	Description
<code>git config --global user.name "<username>"</code>	Configure the author name.
<code>git config --global user.email <email address></code>	Configure the author email address.
<code>git clone ssh://git@github.com/<username>/<repository-name>.git</code>	Create a local copy of a remote repository
<code>git status</code>	Check status
<code>git add <file-name.fileformat></code>	Add a file to the staging area
<code>git add .</code> or <code>git add -A</code>	Add all new and changed files to the staging area
<code>git commit -m "<commit message>"</code>	Commit changes
<code>git rm -r <file-name.fileformat></code>	Remove a file or a folder
<code>git log</code>	View changes
<code>git log --summary</code>	View changes (detailed)
<code>git log --oneline</code>	View changes in one line
<code>git checkout<commit id></code>	Return to previous commit
<code>git revert <commit id></code>	Revert commit (undo one particular commit)
<code>git reset <commit id></code>	Reset to previous commit (remove history of all commit after)
<code>git rm --cached <file/folder></code>	Stop a file being tracked
<code>git checkout <file/to/restore></code>	Restore a file to a previous commit

Table 3. Sample Git commands (continued)

Command	Description
<code>git branch -a</code>	List all branches (local and remote)
<code>git branch <branch name></code>	Create a new branch
<code>git checkout -b <branch name></code>	Create a new branch and switch to it
<code>git checkout -b <branch name> origin/<branch name></code>	Clone a remote branch and switch to it
<code>git branch -m <old branch name> <new branch name></code>	Rename a local branch
<code>git checkout <branch name></code>	Switch to a branch
<code>git checkout -- <file-name.fileformat></code>	Discard changes to a file
<code>git branch -d <branch name></code>	Delete a branch
<code>git push origin --delete <branch name></code>	Delete a remote branch
<code>git diff <source branch> <target branch></code>	Preview changes before merging
<code>git merge <branch name></code>	Merge a branch into the active branch
<code>git merge <source branch> <target branch></code>	Merge a branch into a target branch
<code>git push origin <branch name></code>	Push a branch to your remote repository Push a branch to your remote repository
<code>git push -u origin <branch name></code>	Push changes to remote repository (and remember the branch)
<code>git push</code>	Push changes to remote repository (remembered branch)

Table 3. Sample Git commands (continued)

Command	Description
<code>git push origin --delete <branch name></code>	Delete a remote branch
<code>git pull</code>	Update local repository to the newest commit
<code>git pull origin <branch name></code>	Pull changes from remote repository
<code>git remote add origin ssh://samplewebsite.com/<username>/<repository-name>.git</code>	Add a remote repository

Index

A

abstract 4

S

summary 4